gap-finder-green.vercel.app | Demo: youtu.be/Nxevj7tOB4w

How to evaluate GapFinder in two minutes

One path through the live application — what to **show**, what to **notice**, and why it matters.

	SHOW	NOTICE	WHY IT MATTERS
1	**Open the live app and go to `Check My Work`. Type a worked solution with an error early on — e.g. `2(3x-5) - 4(x+2) = 3(x-1) + 7` expanded so the minus reaches only the first term of the second bracket.**	The screen returns almost immediately and then shows real pipeline stages — reading, verifying, classifying — not a spinner.	The status is polled from actual stage boundaries the orchestrator writes, so the progress is the work, not a timer.
2	**Read the diagnosis. This is the whole product.**	It names the **line number** where the reasoning stopped following — not the final answer. Lines after it are labelled **downstream consequence**, not fresh errors.	Most students circle the wrong line. So do most tools. Four wrong lines become the cost of one idea rather than four failures.
3	**Look at the misconception label and its basis.**	It carries a stable catalogue code and says whether it was `proved` from an algebraic signature or `matched` by a model.	"Proved" and "probably" are stored differently and shown differently. Certainty is not flattened.
4	**Follow the repair: the grounded explanation and the computed diagram, then `Practice`.**	The diagram is drawn from the student's own verified numbers. Every wrong practice option is a documented misconception, not filler.	No image generation touches a maths diagram, and choosing a wrong option is itself informative.
5	**Open `My Learning Gaps`, then `Learning Roadmap`.**	The gap persists with a status (`open / repaired / closed`), and the roadmap orders around it. Only **transfer** closes a gap; practice alone marks it repaired.	This is the difference between feedback and a trajectory. The evidence accumulates in one place.
6	**Open `AI Coach` and ask what you keep getting wrong. Then open `Exam Mode` and answer one question only.**	The Coach cites your own record. Exam Mode returns **`uncertain`** after one answer rather than claiming mastery.	The refusal to over-claim is what makes the positive verdicts worth reading.
7	**`Optional, for technical depth:` open `/dev/observability` — and, if you have the repo, run `npm run verify`.**	Every model call with its stage, provider, cache status, latency and retrieved chunk ids. The verify command passes **with no API keys at all**.	The load-bearing claims are provable offline. That is the whole architectural bet, made checkable.

JUDGING CRITERIA

Evidence against the Devpost rubric

Every claim below names something a judge can open, run, or read in under a minute. Nothing here asks to be taken on trust.

01 · TECHNOLOGICAL IMPLEMENTATION

The hard part is deliberately *not* the model call

Evidence. A deterministic verification layer decides correctness — `mathjs` proportionality checking, per-step-pair routing to algebraic, quantitative or chemical verifiers, and a free-variable guard that returns "could not verify" instead of a verdict. A five-state audit separates a root error from its consequences; misconceptions carry stable codes, derived from an algebraic signature where one exists. A four-stage provider cascade has deterministic substitutes for every stage after vision, and retrieval runs locally with no vector database.

Where to verify. `npm run verify` — lint, typecheck, **228 tests across 14 files**, and a **15-case eval** that runs with no network and no API keys. Live provider behaviour at `/dev/observability`

02 · DESIGN

The interface is shaped by what the system can prove

Evidence. Mobile-first at 390×844, built around a single loop rather than a menu of tools. Certainty is a first-class visual property: `proved` and `matched` read differently; verified, generated and retrieved content are badged distinctly *before* a student reads them; `uncertain` is a designed state rather than an error. Ambiguous handwriting pauses the pipeline and asks, so the student corrects the reading before anything is diagnosed.

Where to verify. The live app at every step of the two-minute path opposite. Document 02 gives purpose, action, response and learning value for all fourteen surfaces

03 · POTENTIAL IMPACT

A misconception is durable; a score is not

Evidence. An arithmetic slip is noise. A misconception is a rule the student keeps applying, so it keeps producing errors until it is named. A **stable code** makes that habit countable across weeks — which turns feedback into a trajectory, and lets Exam Mode hold a returning habit against a mastery claim whatever the score.

Where to verify. `/gaps` shows the recurring-code fingerprint, `/reports/full` consolidates mastery, `/roadmap` orders from that record.

04 · QUALITY OF THE IDEA

Locating the divergence is a different problem from grading the answer

Evidence. In a six-line attempt with one conceptual error at line two, five lines are wrong and *four are not mistakes* — they are consequences of one broken idea. Separating AI interpretation from deterministic verification architecturally is what makes the specific accusation ("line 2") safe to make.

Where to verify. The eval harness measures exactly this: **13 scored cases, 0 failed, 100% divergence accuracy** — including a case of valid work by an unusual route that must *not* be flagged.

DIFFERENTIATION

Four things GapFinder is often mistaken for

Each of these is a real category with real products in it. The distinction is not that GapFinder does more — it is that it answers a different question.

A traditional answer checker

What it does. Compares the final answer to a key and returns correct or wrong, sometimes with a model solution to compare against.

What it cannot do. Tell the student which of their own moves was the bad one. It grades the destination; the student already knew they did not arrive.

A generic AI chatbot

What it does. Answers questions fluently and can explain almost anything on request.

What it cannot do. Be trusted to say a specific line is wrong. It has no verification layer, no stable error vocabulary, and no memory of what this student keeps getting wrong.

A homework solver

What it does. Reads a problem and produces a full worked solution, fast.

What it cannot do. Leave the student able to do the next one. It replaces the attempt rather than repairing it — and an attempt that never happened cannot be diagnosed.

A static learning platform

What it does. Delivers a well-ordered curriculum with videos, exercises and progress tracking.

What it cannot do. Reorder itself around *this* student's actual reasoning. Its sequence was decided before the student arrived.

What the loop does that none of the four does

IT LOCATES

The first line that stopped following, proved by algebra rather than predicted by a model — so the feedback has an address, and an address can be practised.

IT NAMES

The misconception behind that line, from a fixed catalogue. A stable code is countable across weeks; a model-authored label is not.

IT PROVES

Repair is not claimed from the student agreeing. It is claimed from transfer and exam performance with the help removed, and refused where evidence is thin.

IT REMEMBERS

The result feeds the next diagnosis, the roadmap order and the next exam verdict. The loop closes; the other four end at a verdict.

The honest version of this comparison. GapFinder verifies a narrower slice of mathematics than a large model can discuss — single-variable linear algebra is what is genuinely *proved*. That narrowness is the trade: within it, the product can point at a line and be right, which is precisely what a broader system cannot safely do.

WHY THIS PROJECT MATTERS

The through-line, stated once

Every claim below is one that appears elsewhere in this dossier with a file, a route or a command attached to it.

THE PROBLEM & THE EDUCATION

Feedback without a location is not teachable

"Wrong" tells a student to try harder. "Line 2, and here is the rule you were applying" tells them what to change. Only the second can be practised, re-tested, and eventually closed — and only the second distinguishes a slip from a habit that will reappear next week in a different chapter.

THE AI

Used where it is strong, bounded where it is not

A model reads handwriting and phrases teaching — things nothing else can do. It is never the authority on whether a step is correct, because a confidently wrong accusation is the one failure a tutoring product cannot recover from. The separation is architectural, not a prompt instruction.

THE PERSONALISATION

Built on the learner's own verified record

Stable misconception codes, a computed mastery EMA, and gap status shape what is offered next, what the Coach can cite, and what Exam Mode watches for. Not an inferred learning style, not a trained per-student model — an evidence trail the student can inspect.

THE TECHNICAL EXECUTION

Complete, deployed, and verifiable offline

Not a prototype of one screen: 31 pages, 40 API routes, 30 data models, live on Vercel. 228 tests and the eval harness pass with **no API keys configured** — the load-bearing claims do not depend on a model being reachable.

THE USER EXPERIENCE

Certainty is visible, not flattened

Proved reads differently from matched; verified, generated and retrieved content are badged before they are read; ambiguity pauses the pipeline and asks; `uncertain` is a designed state. The interface can be specific precisely because the system knows when it is not.

THE FUTURE POTENTIAL

The extension surface is the narrow part

A new subject needs a verifier, a routing rule, catalogue entries and seeded chunks — the loop around it is unchanged. A new AI or resource provider implements one interface and inherits the cascade, caching, logging and fallback. Widening the verified slice widens the product.

A wrong answer is a symptom. GapFinder finds the line where the reasoning diverged, proves it, names the habit behind it, and refuses to call it fixed until it holds without help.

13/13
scored eval cases — 100% divergence accuracy

228
tests passing with no API keys

17
stable misconception codes across four subjects

1
rule the whole system obeys: the AI interprets, the code verifies

The strongest verified thing this project can say about itself is not that it is clever. It is that **when it cannot check a line, it says so** — and that everything it does claim can be re-run, on any machine, without a network. `npm run verify`